

Java

Strings, class & object

Theory Notes

1. Strings in Java

- **Immutable** → once created, cannot be changed.
- Stored in **String Constant Pool (SCP)** for memory efficiency.
- Comparison:
 - `==` → compares references.
 - `.equals()` → compares values.

2. String Methods

- `length()`, `charAt()`, `indexOf()`, `lastIndexOf()`
- `toUpperCase()`, `toLowerCase()`, `trim()`
- `equalsIgnoreCase()`, `contains()`, `substring()`, `replace()`
- `startsWith()`, `endsWith()`, `concat()`

3. StringBuilder vs StringBuffer

- **StringBuilder** → mutable, non-synchronized (faster).
- **StringBuffer** → mutable, synchronized (thread-safe).
- Common methods: `append()`, `insert()`, `delete()`, `reverse()`.

4. Classes & Objects

- Class → blueprint.
- Object → instance of class.
- Fields → variables inside class.
- Methods → behavior of class.

Coding Round Questions

1. Create a string "FIIT" and compare it with another string using both `==` and `.equals()`.
2. Demonstrate `substring()` by extracting "FI" from "FIIT".
3. Write a program to reverse a string using `StringBuilder`.
4. Show the difference between `StringBuilder` and `StringBuffer` by appending text in both.
5. Create a class `Employee` with a field `salary` and a method `display()`. Create two objects and print their details.
6. Demonstrate `startsWith()` and `endsWith()` methods on "FIIT".

Challenge

Problem: Student Report System

Requirements:

- Create a class `Student` with fields `name`, `marks`.
- Add methods to calculate grade (A, B, C) based on marks.
- Store multiple students in an array.
- Use `StringBuilder` to build a formatted report.
- Print the report with student names, marks, and grades.

```
class Employee{
    int salary = 50000; // field

    void display(){
        System.out.println("Employee display method");
    }
}

public class StringOps {

    public static void main(String[] args) {

        Employee obj = new Employee();
        Employee obj1 = new Employee();

        System.out.println(obj.salary);
        obj.display();

        System.out.println(obj1.salary);
        obj1.display();

        // System.out.println("Hello there");

        // // string -> sequence of character

        // String name = "FIIT"; // non-primitive (heap)

        // String name1 = "FIIT"; //string literal (scp)

        // System.out.println(name==name1); // == checks for reference

        // String name2 = new String("fiit"); //create new object

        // System.out.println(name2);
```

```
// System.out.println(name == name2); //

// System.out.println(name.equals(name2)); // checks only for value

// // string -> immutable (unchangeable)

// // string methods
// System.out.println(name.length());
// System.out.println(name.charAt(3));
// System.out.println(name.indexOf('I'));

// System.out.println(name.lastIndexOf('I'));

// System.out.println(name.toUpperCase());
// System.out.println(name.toLowerCase());
// System.out.println(name.trim());

// System.out.println(name.equals(name2));

// System.out.println(name.equalsIgnoreCase(name2));
// System.out.println(name.contains("FI"));

// System.out.println(name.substring(0,3));
// // substring -> n-1 -> 3-1 -> 2

// System.out.println(name.replace("II", "AA"));
// System.out.println(name); // FIIT OR FAAT

// System.out.println(name.startsWith("F"));
// System.out.println(name.endsWith("IIT"));

// System.out.println(name.concat(name2));


// string builder vs string buffer

// string builder -> mutable , non-synchronized

StringBuilder sb = new StringBuilder("Academy");
sb.append("FIIT");
sb.insert(7, " ");
sb.delete(0, 7);

System.out.println(sb);
```

```
// string buffer -> mutable , synchronized

StringBuffer sf = new StringBuffer("Coursee");
sf.append("FIIT");
sf.insert(7, " ");
sf.delete(0, 7);
System.out.println(sf);
// javascript -> single-threaded
// process vs thread

// class and object

}

}
```